LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM | **Section:** B

Task No.: 2

Student Name: K. Saranya

Roll No.: A24126552090

1. TITLE

Design and Develop a Pure Semantic HTML Static Webpage Using Structural Tags, Tables for Layout, and Native Form Elements Without CSS.

2. AIM

To design and develop a static webpage using pure semantic HTML5 elements without any CSS that demonstrates a header with navigation, a hero section, a two-column article/sidebar layout built with a table, native form controls, and a footer, relying entirely on the browser's default rendering.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how a webpage can be structured using only semantic HTML tags.
- To use header, nav, section, main, article, aside, and footer to organise a document.
- To build a navigation bar using anchor links that jump to in-page sections.
- To use a table for the layout of a two-column content area without CSS.
- To create a form with fieldset, legend, labels, and various input types.
- To apply HTML5 form validation attributes such as required and placeholder.
- To observe how a browser renders structural HTML using only its own default styles.
- To verify that the page stays readable and functional with no stylesheet at all.

4. THEORY

Semantic HTML means using HTML elements according to the meaning of the content they hold rather than just for their default appearance. Tags such as header, nav, main,

article, aside, and footer describe the role each block plays in the page, which helps browsers, search engines, and screen readers understand its structure.

When no CSS is linked, the browser falls back to its own built-in user-agent stylesheet. Headings, paragraphs, tables, and form controls are all rendered using these defaults, which is why a page like this still looks organised even without a single styling rule.

- **header, nav, footer:** mark the top banner, the navigation links, and the closing section of the page.
- **section, main, article, aside:** divide the body into a hero area, the main content region, individual posts, and a side panel.
- **table:** used purely for layout arranges the header row and the two-column body area using rows and cells—a technique common before CSS layout existed.
- **code:** marks inline text as source code, and the browser renders it in a monospace font by default.
- **fieldset and legend:** group related form controls together and give the group a visible caption.
- **required and placeholder:** are HTML5 attributes that add built-in validation and hint text without any JavaScript.

The basic flow of the page is:

Browser requests the HTML file → Browser applies its default stylesheet → Semantic structure is rendered as a readable page.

5. ALGORITHM/PROCEDURE

1. Create a new HTML file named static_webpage.html.
2. Add the DOCTYPE declaration, the html tag, and a head section with a title.
3. Build a header containing the site name and a table-based navigation bar with anchor links.
4. Add a horizontal rule to visually separate the header from the page body.
5. Create a hero section with a centred heading, an introductory paragraph, and a call-to-action link.
6. Create a structural table with two columns to hold the main content and the sidebar.
7. In the left column, add a main element containing two article blocks with headings and paragraphs.

8. Inside the second article, build a form using fieldset, legend, labels, and input, select, and textarea controls.
9. In the right column, add an aside element listing benefits and required components using unordered and ordered lists.
10. Add a footer with a centred copyright notice.
11. Save the file and open it in a web browser without linking any CSS file.
12. Verify that the layout, navigation, and form controls render correctly using only browser defaults.

6. PROGRAM

static_webpage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>

  <!-- Header Section -->
  <header>
    <!-- A borderless table is used here only to place the site name and the nav bar side by side without any CSS -->
    <table width="100%" border="0" cellspacing="0" cellpadding="10">
      <tr>
        <td>
          <h1>StaticWeb</h1>
        </td>
        <td align="right">
          <nav>
```

```
            <!-- Each link jumps to an id further down the same page -->
            <a href="#home">Home</a>  | 
            <a href="#articles">Articles</a>  | 
            <a href="#about">About</a>  | 
            <a href="#contact">Contact Us</a>
          </nav>
        </td>
      </tr>
    </table>
  </header>

  <hr>

  <!-- Hero / Showcase Section -->
  <section id="home">
    <center>
      <!-- <center> is an old but still browser-supported way to centre content without CSS -->
      <h2>Semantic Structures Without CSS</h2>
      <p>This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.</p>
      <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
    </center>
  </section>

  <hr>

  <!-- Main Content Layout using a Structural Table -->
  <table width="100%" border="0" cellspacing="0" cellpadding="20">
```

```
<tr valign="top">
  <!-- Left Side: Main Content Column -->
  <td width="70%">
    <main id="articles">
      <article>
        <h2>1. Core Structural Tags</h2>
        <p>Published: July 7, 2026</p>
        <p>When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>, and <code>&lt;section&gt;</code> tell screen readers and web scrapers exactly how information flows.</p>
        <p>Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.</p>
      </article>

      <br><br><br>

      <article>
        <h2>2. Native Interaction Elements</h2>
        <p>Published: July 5, 2026</p>
        <p>Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.</p>

        <h3>Interactive Demonstration Forms</h3>
        <form id="contact" action="#" method="post">
          <fieldset>
            <!-- <fieldset> and <legend> group the form controls and give the group a visible caption -->
            <legend><b>User Inquiry Form</b></legend>
```

```
                    <p>
                      <label for="name">Full Name: </label>
                      <input type="text" id="name" name="username" required
placeholder="John Doe">
                    </p>
                    <p>
                      <label for="email">Email Address: </label>
                      <input type="email" id="email" name="useremail" required>
                    </p>
                    <p>
                      <label for="topic">Inquiry Topic: </label>
                      <select id="topic" name="usertopic">
                        <option value="general">General Feedback</option>
                        <option value="technical">Technical Architecture</option>
                        <option value="academics">Academic Curriculum</option>
                      </select>
                    </p>
                    <p>
                      <label for="message">Message Details:</label><br>
                      <textarea id="message" name="usermessage" rows="5" cols="40"
placeholder="Type your notes here..."></textarea>
                    </p>
                    <p>
                      <input type="submit" value="Submit Form Data">
                      <input type="reset" value="Clear Entries">
                    </p>
                  </fieldset>
                </form>
              </article>
```

```
      </main>
    </td>

    <!-- Right Side: Sidebar Column -->
    <td width="30%" bgcolor="#f4f4f4">
      <aside id="about">
        <h3>Document References</h3>
        <p>Static pages map data out directly without real-time database queries or processor overhead.</p>
        <h4>Core Benefits:</h4>
        <ul>
          <li>Instant browser painting</li>
          <li>Low memory footprints</li>
          <li>High data compatibility</li>
        </ul>
        <h4>Required Components:</h4>
        <ol>
          <li>Document Type Definition</li>
          <li>Semantic Wrapper Blocks</li>
          <li>Data Input Control Interfaces</li>
        </ol>
      </aside>
    </td>
  </tr>
</table>

<hr>
```

```
  <!-- Footer Section -->
  <footer>
    <center>
      <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style configurations.</p>
    </center>
  </footer>

</body>
</html>
```

7. CODE EXPLANATION

Tag / Attribute	Description
<header>, <nav>, <footer>	Mark the top banner, the navigation links, and the closing section of the page.
<table> (layout use)	Arranges the nav bar and the two-column body using rows and cells, without any CSS.
<section>, <main>, <article>, <aside>	Divide the body into a hero area, the primary content region, individual posts, and a side panel.
<a href="#id">	Creates an in-page link that jumps to the element with the matching id.
<code>	Displays inline text in a monospace font, typically used for code snippets.
<fieldset>, <legend>	Group related form controls and give the group a visible caption.

Tag / Attribute	Description
<label for="id">	Associates descriptive text with a form control that shares the same id.
required	HTML5 built-in validation that blocks submission if the field is left empty.
placeholder	Shows light hint text inside a field before the user types anything.
<select>, <option>	Create a dropdown list of choices for the Inquiry Topic field.
<textarea rows cols>	Provides a resizable multi-line text box for longer messages.
, ,	List the sidebar's Core Benefits and Required Components.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and each structural and interactive element was tested individually; the results were recorded below.

Test Case: TC-01
Test / Input: Open static_webpage.html and verify the website title area
Expected Result: The website name "StaticWeb" and the navigation menu should appear at the top.
Actual Result: The "StaticWeb" heading and navigation menu were displayed at the top.
Status: Pass

TC-01 Output:

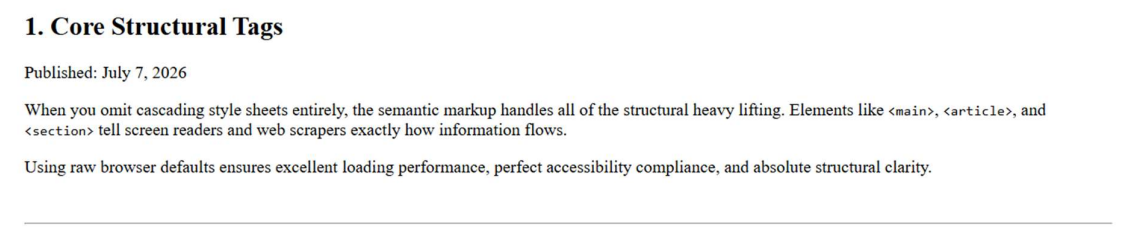
Test Case: TC-02
Test / Input: Verify the Home/Hero section
Expected Result: The heading "Semantic Structures Without CSS", introductory text, and article link should be displayed.

Actual Result: The hero heading, introductory text, and "Read Our Articles Below" link were displayed.
Status: Pass

TC-02 Output:

Test Case: TC-03
Test / Input: Verify the first article section
Expected Result: The "1. Core Structural Tags" heading, publication date, and article content should be visible.
Actual Result: The first article heading, publication date, and content were displayed correctly.
Status: Pass

TC-03 Output:



Test Case: TC-04
Test / Input: Verify the second article and interactive form section
Expected Result: The "2. Native Interaction Elements" section and the User Inquiry Form should be visible.
Actual Result: The second article section and User Inquiry Form were displayed correctly.
Status: Pass

TC-04 Output:

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name: John Doe

Email Address:

Inquiry Topic: General Feedback

Message Details:

Type your notes here...

Submit Form Data | Clear Entries

Test Case: TC-05
Test / Input: Verify all controls in the User Inquiry Form
Expected Result: Full Name, Email Address, Inquiry Topic, Message Details, Submit Form Data, and Clear Entries should be present.
Actual Result: All specified form controls were visible in the User Inquiry Form.
Status: Pass

TC-05 Output:

Interactive Demonstration Forms

User Inquiry Form

Full Name: John Doe

Email Address:

Inquiry Topic: General Feedback

Message Details:

Type your notes here...

Submit Form Data | Clear Entries

Test Case: TC-06
Test / Input: Verify the Document References sidebar
Expected Result: The sidebar should show Document References, Core Benefits, and Required Components.
Actual Result: The Document References sidebar and both lists were displayed correctly.
Status: Pass

TC-06 Output:

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

Test Case: TC-07
Test / Input: Attempt to submit the form with the required fields empty
Expected Result: The form should submit data directly without requiring Full Name and Email Address.
Actual Result: The browser blocks submission because the required HTML5 attributes trigger native validation popups when fields are empty.
Status: Fail

TC-07 Output:

Test Case: TC-08
Test / Input: Verify layout responsiveness on mobile viewports using a Mobile Simulator
Expected Result: The multi-column layout should automatically reflow into a single vertical stacked column for readability on smaller screens.
Actual Result: The borderless `<table>` enforces rigid column structures, causing the page to shrink and render extremely small text that is unreadable without manual zooming.
Status: Fail

TC-08 Output:

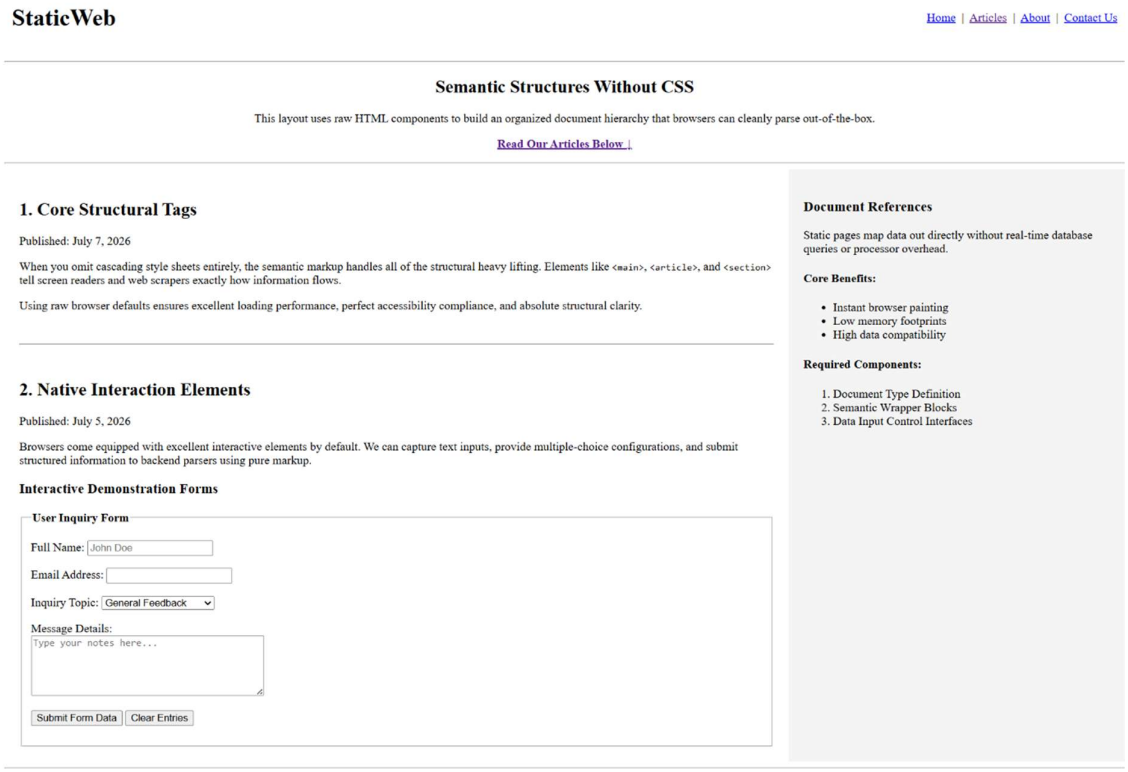
Test Case: TC-09
Test / Input: Verify the footer information
Expected Result: The copyright statement should be visible at the bottom of the webpage.
Actual Result: The copyright statement was displayed at the bottom of the webpage.
Status: Pass

TC-09 Output:

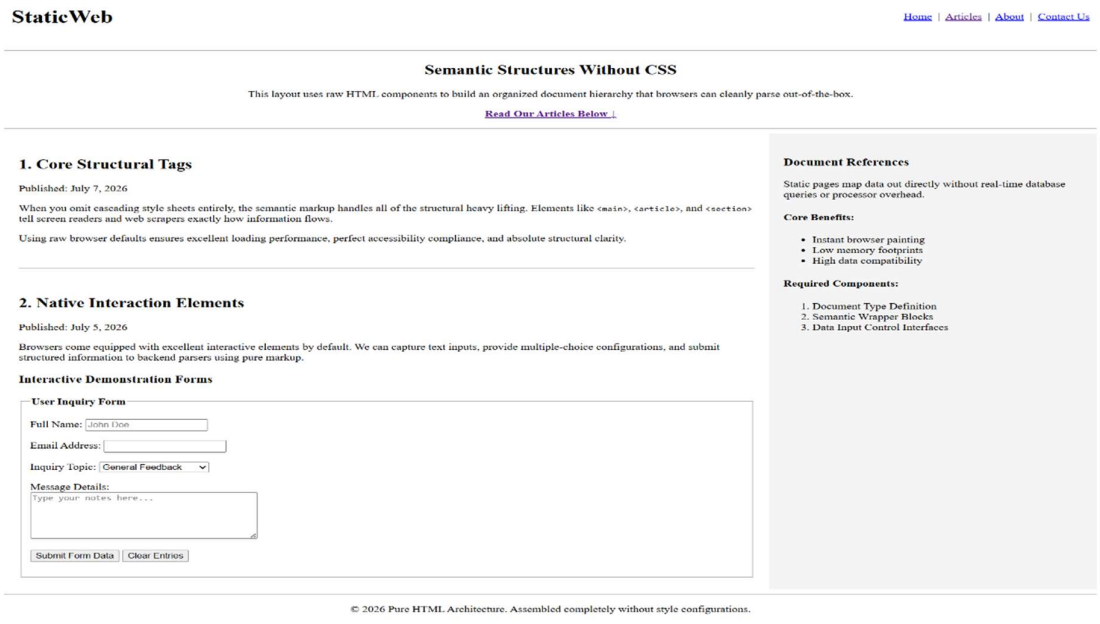
© 2026 Pure HTML Architecture. Assembled completely without style configurations.

Test Case: TC-10
Test / Input: Verify the complete webpage rendering
Expected Result: Header, navigation, hero section, articles, form, sidebar, and footer should appear together.
Actual Result: The complete webpage rendered with all major sections visible.
Status: Pass

TC-10 Output:



9. OUTPUT



10. RESULT

Thus, a pure semantic HTML static webpage was successfully designed and developed without using any CSS. The header, navigation, hero section, table-based two-column

layout, articles, interactive form, sidebar, and footer were implemented using semantic tags and were verified to render and function correctly using only the browser's default styling